


**Πανεπιστήμιο Δυτικής Αττικής**
**Σχολή Μηχανικών**
**Τμήμα Μηχανικών Πληροφορικής και Υπολογιστών**

# ΤΕΧΝΟΛΟΓΙΑ ΚΑΙ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ ΚΙΝΗΤΩΝ ΣΥΣΚΕΥΩΝ

**ΠΑΠΑΓΕΩΡΓΙΟΥ ΦΙΛΙΠΠΟΣ - ΑΜ: 21390174**
**ΘΩΜΑΣ ΝΙΚΟΛΑΟΣ - ΑΜ: 21390068**

**ΑΘΗΝΑ**

# Πίνακας Περιεχομένων

1. Εισαγωγή ..... 3
1.1 Σκοπός της Εργασίας ..... 3
1.2 Περιγραφή του Προβλήματος ..... 3
1.3Προσέγγιση Λύσης ..... 3
1.4 Τεχνολογίες που Χρησιμοποιήθηκαν ..... 3
2. Ανάλυση Απαιτήσεων ..... 4
2.1 Απαιτήσεις από την Εκφώνηση ..... 4
2.2 Λειτουργικές Απαιτήσεις ..... 4
2.3 Τεχνικές Προδιαγραφές ..... 5
3. Αρχιτεκτονική Συστήματος ..... 5
3.1Γενική Αρχιτεκτονική ..... 5
4. Υλοποίηση Android Εφαρμογής ..... 7
4.1 Δομή Project και Κλάσεις ..... 7
4.2 MainActivity - Κύρια Λειτουργικότητα ..... 7
4.3Network Scanning και TCP Communication ..... 7
4.4UI/UX Σχεδιασμός με RecyclerView ..... 8
4.5Wake-on-LAN Υλοποίηση ..... 8
4.6Υλοποίηση TCP Server ..... 9
5. Τρόπος Λειτουργίας και Screenshots ..... 9
5.3 Wake-on-LAN Λειτουργικότητα ..... 11
6. Δοκιμές και Αποτελέσματα ..... 11
6.1 Περιβάλλον Δοκιμών ..... 11

# 1. Εισαγωγή

## 1.1 Σκοπός της Εργασίας

Η παρούσα εργασία στοχεύει στην ανάπτυξη ενός ολοκληρωμένου συστήματος διαχείρισης και παρακολούθησης των υπολογιστών του εργαστηρίου Κ10.018. Το σύστημα αποτελείται από μια Android εφαρμογή που επικοινωνεί με TCP servers που εκτελούνται σε κάθε εργαστηριακό υπολογιστή.

## 1.2 Περιγραφή του Προβλήματος

Στο εργαστήριο Κ10.018 λειτουργούν 27 υπολογιστές οι οποίοι μπορούν να τρέχουν είτε OpenSuse Linux είτε Windows 10. Χρειάζεται να υλοποιηθεί:

- **Απομακρυσμένη διαχείριση** των υπολογιστών μέσω κινητής συσκευής
- **Παρακολούθηση κατάστασης** (online/offline, λειτουργικό σύστημα)
- **Εκτέλεση βασικών εντολών** όπως επανεκκίνηση, τερματισμός, επαναφορά
- **Εκκίνηση κλειστών υπολογιστών** μέσω Wake-on-LAN
- **Κεντρικός έλεγχος** από ένα εύχρηστο interface

## 1.3 Προσέγγιση Λύσης

Η λύση που υλοποιήθηκε περιλαμβάνει:

**Android Εφαρμογή (Client):**

- Σάρωση δικτύου για εντοπισμό διαθέσιμων υπολογιστών
- Αποστολή εντολών μέσω TCP connections
- Εμφάνιση real-time status των υπολογιστών
- Wake-on-LAN λειτουργικότητα

**TCP Server:**

- Εκτέλεση σε κάθε εργαστηριακό υπολογιστή
- Listening στην πόρτα 41007
- Χειρισμός εντολών Echo, Restart, Shutdown, Restore
- Επιστροφή κατάστασης και πληροφοριών συστήματος

## 1.4 Τεχνολογίες που Χρησιμοποιήθηκαν

- **Γλώσσα Προγραμματισμού:** Java

- **Platform:** Android SDK (API 26+)
- **Δικτύωση:** TCP Sockets, Wake-on-LAN Protocol
- **UI Framework:** Android XML Layouts, RecyclerView
- **Build System:** Gradle

# 2. Ανάλυση Απαιτήσεων

## 2.1 Απαιτήσεις από την Εκφώνηση

Με βάση την εκφώνηση της εργασίας, το σύστημα πρέπει να υλοποιεί τις ακόλουθες λειτουργίες:

**Βασικές Απαιτήσεις:**

- Ανάπτυξη Android εφαρμογής σε γλώσσα Java
- Επικοινωνία με TCP servers στην πόρτα 41007
- Υποστήριξη 27 υπολογιστών (PRPC01 έως PRPC27)
- Λειτουργία εντός LAN του εργαστηρίου

**Υποστηριζόμενες Εντολές:**

- **Echo**: Επιστροφή hostname και λειτουργικού συστήματος
- **Restart**: Μήνυμα επανεκκίνησης υπολογιστή
- **Shutdown**: Μήνυμα Τερματισμού λειτουργίας
- **Restore**: Μήνυμα επαναφοράς (60-120 δευτερόλεπτα)

## 2.2 Λειτουργικές Απαιτήσεις

**Επιλογή και Αποστολή Εντολών**

- Ο χρήστης πρέπει να μπορεί να επιλέγει μία εντολή
- Να την στέλνει σε έναν ή περισσότερους υπολογιστές
- Να λαμβάνει επιβεβαίωση εκτέλεσης

**Ανάγνωση Μηνυμάτων Server**

- Διαβάζει και εμφανίζει τα μηνύματα από τους servers
- Διαμόρφωση των μηνυμάτων για καλύτερη παρουσίαση

**Έλεγχος Κατάστασης Υπολογιστών**

- Παρακολούθηση ποιοί υπολογιστές είναι σε λειτουργία
- Αναγνώριση λειτουργικού συστήματος (Linux/Windows10)
- Real-time ενημέρωση status

**Wake-on-LAN (WOL)**

- Εκκίνηση κλειστών υπολογιστών
- Αποστολή magic packet με MAC address

## 2.3 Τεχνικές Προδιαγραφές

**Δίκτυο:**

- Πρωτόκολλο :TCP/IP
- Πόρτα Server:41007
- Εύρος IP: 192.168.88.2 - 192.168.88.28
- Όνομα: PRPCXX (XX -> 01-27)

**Android Απαιτήσεις:**

- Minimum SDK: API 26 (Android 8.0)
- Target SDK: API 35
- Permissions : INTERNET, ACCESS_NETWORK_STATE, ACCESS_WIFI_STATE

**Server Αρχιτεκτονική:**

- Daemon process που εκκινεί με το σύστημα Listening mode στην πόρτα 41007
- Χειρισμός concurrent connections Cross-platform (Linux/Windows)

# 3. Αρχιτεκτονική Συστήματος

## 3.1 Γενική Αρχιτεκτονική

Το Caller Application ακολουθεί την κλασική αρχιτεκτονική Client-Server όπου η Android εφαρμογή λειτουργεί ως client και κάθε εργαστηριακός υπολογιστής φιλοξενεί έναν TCP server. Η επικοινωνία πραγματοποιείται αποκλειστικά μέσω του τοπικού δικτύου του εργαστηρίου. Κάθε server εκτελείται ως daemon process και παραμένει ενεργός καθόλη τη διάρκεια λειτουργίας του υπολογιστή, ακούγοντας συνεχώς την πόρτα 41007 για εισερχόμενες εντολές. Η Android εφαρμογή διαθέτει μηχανισμό

αυτόματης ανίχνευσης των διαθέσιμων servers μέσω network scanning, στέλνοντας Echo εντολές σε όλες τις προκαθορισμένες IP διευθύνσεις. Όταν ένας server ανταποκρίνεται, η εφαρμογή καταγράφει την κατάστασή του και τον προσθέτει στη λίστα ενεργών υπολογιστών. Για τους υπολογιστές που δεν ανταποκρίνονται, το σύστημα διατηρεί τις πληροφορίες τους (IP, MAC address) προκειμένου να είναι δυνατή η αποστολή Wake-on-LAN packets για την εκκίνησή τους.

## 3.2 Διάγραμμα Επικοινωνίας

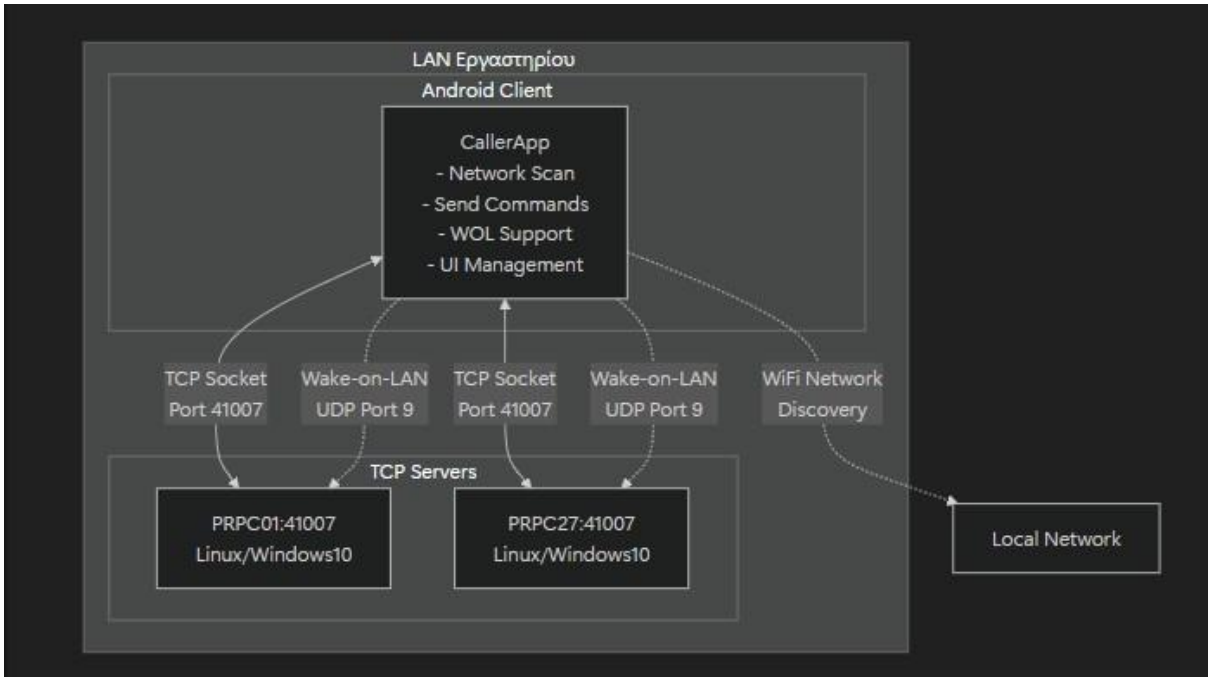



***Diagram 1***

## 3.3 Τεχνολογίες και Πρωτόκολλα

Η υλοποίηση βασίζεται σε τρία κύρια τεχνολογικά στοιχεία. Πρώτον, η TCP socket επικοινωνία που εξασφαλίζει μετάδοση των εντολών και των απαντήσεων μεταξύ client και server. Δεύτερον, το Wake-on-LAN πρωτόκολλο που επιτρέπει την απομακρυσμένη εκκίνηση υπολογιστών μέσω της αποστολής "magic packets" που περιέχουν το MAC address του στόχου. Τρίτον, η Android networking API που διαχειρίζεται τις network operations σε background threads για να αποφευχθεί το blocking του UI thread. Η αρχιτεκτονική υιοθετεί το μοντέλο του stateless server, όπου κάθε εντολή αποτελεί μια ανεξάρτητη συναλλαγή χωρίς διατήρηση κατάστασης μεταξύ των requests. Αυτό επιτρέπει την ταυτόχρονη εξυπηρέτηση πολλαπλών clients.

# 4. Υλοποίηση Android Εφαρμογής

## 4.1 Δομή Project και Κλάσεις

Η Android εφαρμογή CallerApplication αναπτύχθηκε με σύγχρονες αρχές αρχιτεκτονικής Android και οργανώθηκε σε διακριτές κλάσεις με συγκεκριμένες ευθύνες. Η εφαρμογή χρησιμοποιεί το namespace "org.nifi.callerapplication" και στοχεύει σε Android devices με minimum SDK 26 και target SDK 35, εξασφαλίζοντας συμβατότητα με σύγχρονες συσκευές. Οι κύριες κλάσεις της εφαρμογής είναι η Computer που αναπαριστά το μοντέλο δεδομένων κάθε υπολογιστή, η MainActivity που διαχειρίζεται την κύρια λειτουργικότητα και το UI, η ComputerAdapter που υλοποιεί το RecyclerView pattern για την εμφάνιση της λίστας υπολογιστών, η ComputerList που περιέχει τις προκαθορισμένες πληροφορίες των εργαστηριακών υπολογιστών, και η WolUtil που υλοποιεί τη λειτουργικότητα Wake-on-LAN. Κάθε κλάση έχει σχεδιαστεί να είναι αυτόνομη και να ακολουθεί το Single Responsibility Principle.

## 4.2 MainActivity - Κύρια Λειτουργικότητα

Η MainActivity αποτελεί το κεντρικό σημείο ελέγχου της εφαρμογής και υλοποιεί όλες τις βασικές λειτουργίες που απαιτούνται από την εκφώνηση. Κατά την εκκίνηση, η activity αρχικοποιεί το RecyclerView με την προκαθορισμένη λίστα υπολογιστών και ξεκινά αυτόματα μια διαδικασία network scanning για τον εντοπισμό ενεργών servers. Η κλάση διαχειρίζεται το StrictMode policy για να επιτρέπει network operations στο main thread όταν είναι απαραίτητο, αν και οι περισσότερες δικτυακές λειτουργίες εκτελούνται σε background threads. Η μέθοδος scanNetwork() υλοποιεί έναν αποδοτικό αλγόριθμο που διατρέχει όλους τους προκαθορισμένους υπολογιστές και δοκιμάζει TCP connection στην πόρτα 41007 με timeout 500ms. Για κάθε επιτυχή σύνδεση, στέλνει μια "Echo" εντολή και αναλύει την απάντηση για να εξάγει το hostname και το λειτουργικό σύστημα. Η testConnection() μέθοδος διαχειρίζεται τις exceptions και ενημερώνει το UI thread μέσω runOnUiThread() για να αποφευχθούν concurrency issues. Επιπλέον, η MainActivity υλοποιεί τη sendWakeOnLan() μέθοδο που επιτρέπει την εκκίνηση offline υπολογιστών με χρήση των MAC addresses που είναι αποθηκευμένα στη ComputerList.

## 4.3 Network Scanning και TCP Communication

Η εφαρμογή υλοποιεί ένα γρήγορο σύστημα network discovery. Η στρατηγική που ακολουθείται περιλαμβάνει τη δημιουργία ενός Socket instance για κάθε υπολογιστή με συγκεκριμένο timeout, τη χρήση του InetSocketAddress για precise connection targeting, και την άμεση αποστολή Echo command μετά την επιτυχή σύνδεση. Όταν ένας server ανταποκρίνεται, η εφαρμογή αναλύει την απάντηση που

έχει τη μορφή "HOSTNAME - OS" και ενημερώνει τα αντίστοιχα πεδία του Computer object. Η TCP επικοινωνία διαχειρίζεται μέσω BufferedReader και PrintWriter instances που εξασφαλίζουν αποδοτική μετάδοση δεδομένων. Η εφαρμογή υλοποιεί proper resource management με try-with-resources statements για αυτόματο κλείσιμο των connections και αποφυγή memory leaks. Ιδιαίτερη προσοχή δόθηκε στο error handling, όπου κάθε IOException ή timeout εξαιρέσεις καταγράφονται και ο αντίστοιχος υπολογιστής σημειώνεται ως offline, χωρίς να διακόπτεται η διαδικασία scanning για τους υπόλοιπους.

## 4.4 UI/UX Σχεδιασμός με RecyclerView

Η διεπαφή χρήστη σχεδιάστηκε με έμφαση στην απλότητα και τη λειτουργικότητα, ακολουθώντας τις Material Design guidelines του Android. Το κύριο UI component είναι ένα RecyclerView που εμφανίζει τη λίστα όλων των υπολογιστών με real-time status updates. Κάθε item στη λίστα περιλαμβάνει έναν χρωματικό indicator (πράσινος για online, κόκκινος για offline), την IP διεύθυνση, πληροφορίες για το hostname και λειτουργικό σύστημα, καθώς και text-based status indicator. Η χρήση LinearLayoutManager εξασφαλίζει smooth scrolling και efficient memory usage ακόμα και με μεγάλο αριθμό items. Το ComputerAdapter υλοποιεί intelligent click handling που διαφοροποιεί τη συμπεριφορά ανάλογα με την κατάσταση κάθε υπολογιστή. Για online υπολογιστές, το click εμφανίζει ένα AlertDialog με τις διαθέσιμες εντολές (Echo, Restart, Shutdown, Restore), ενώ για offline υπολογιστές προτείνει την αποστολή Wake-on-LAN packet. Η visual feedback παρέχεται μέσω alpha transparency για offline items, δημιουργώντας εύκολα αναγνωρίσιμη διαφοροποίηση κατάστασης. Το Rescan button στο top της οθόνης επιτρέπει manual refresh του network status όποτε ο χρήστης το επιθυμεί.

## 4.5 Wake-on-LAN Υλοποίηση

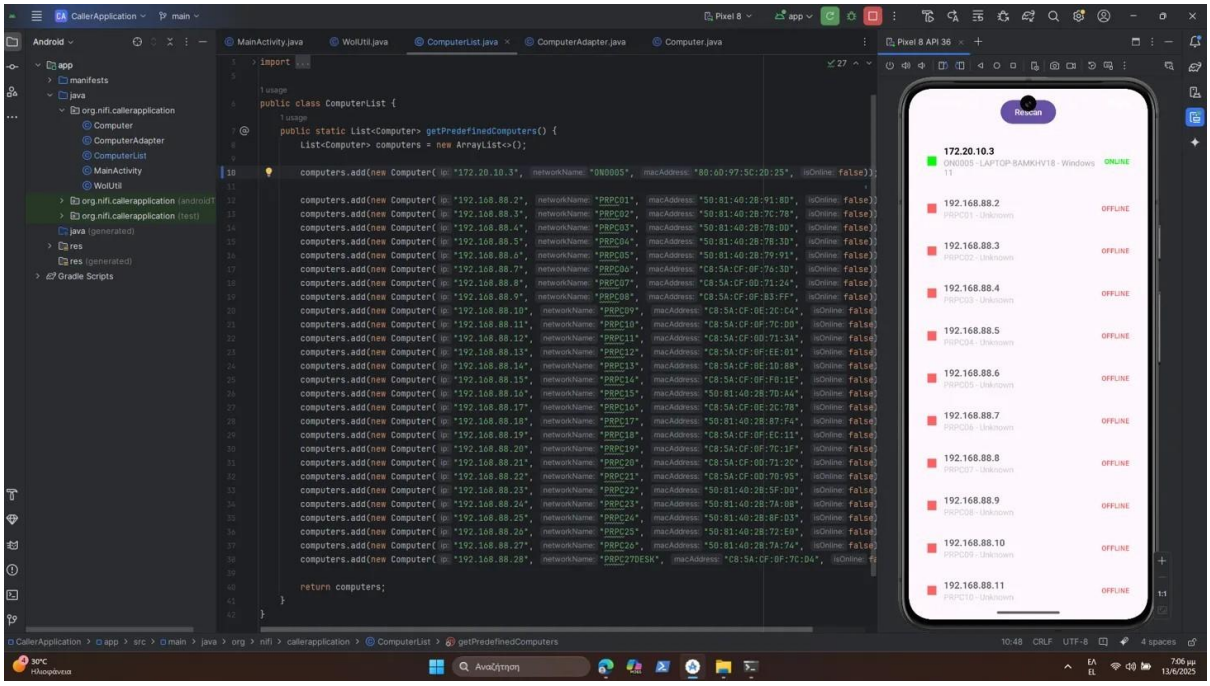
Η υλοποίηση του Wake-on-LAN πρωτοκόλλου στην WolUtil κλάση αποτελεί ένα από τα πιο τεχνικά προκλητικά μέρη της εφαρμογής. Η κλάση υλοποιεί τη δημιουργία του "magic packet" που αποτελείται από έξι bytes με τιμή 0xFF ακολουθούμενα από 16 επαναλήψεις του MAC address του στόχου. Η μέθοδος getMacBytes() αναλύει το MAC address από string format σε byte array, υποστηρίζοντας τόσο colon όσο και hyphen separators για maximum compatibility. Ο αλγόριθμος διαχειρίζεται validation errors και παρέχει descriptive exception messages για debugging σκοπούς. Η αποστολή του WOL packet γίνεται μέσω UDP DatagramSocket στην πόρτα 9, με στόχο τη broadcast διεύθυνση του subnet. Η εφαρμογή υπολογίζει αυτόματα τη broadcast διεύθυνση από την IP του target υπολογιστή, αντικαθιστώντας το τελευταίο octet με 255. Αυτή η προσέγγιση εξασφαλίζει ότι το packet θα φτάσει στον target υπολογιστή ανεξάρτητα από τη δικτυακή τοπολογία του εργαστηρίου. Η υλοποίηση περιλαμβάνει comprehensive error handling και logging για να διευκολύνει την αντιμετώπιση προβλημάτων κατά τη λειτουργία.

## 4.6 Υλοποίηση TCP Server

Ο TCP Server υλοποιήθηκε ως απλό Java application που εκτελείται ως daemon process σε κάθε εργαστηριακό υπολογιστή. Ο server δημιουργεί ένα ServerSocket instance που ακούει συνεχώς την πόρτα 41007 και δέχεται concurrent connections από την Android εφαρμογή. Κατά την εκκίνηση, ο server αποκτά αυτόματα το hostname και το λειτουργικό σύστημα του υπολογιστή μέσω των System properties, πληροφορίες που χρησιμοποιούνται για την απάντηση στην Echo εντολή. Η λειτουργία του server βασίζεται σε έναν απλό request-response μηχανισμό όπου κάθε incoming connection διαχειρίζεται σε blocking mode. Όταν λαμβάνεται μια εντολή, ο server την αναλύει μέσω switch statement και επιστρέφει την κατάλληλη απάντηση. Για την εντολή "Echo" επιστρέφει το format "HOSTNAME - OS", για "Restart" και "Shutdown" επιστρέφει confirmation messages, ενώ για "Restore" προσομοιώνει μια χρονοβόρα διαδικασία με Thread.sleep() και στέλνει δύο μηνύματα (Restoring και Restored) όπως απαιτεί η εκφώνηση. Η υλοποίηση περιλαμβάνει comprehensive error handling και logging για monitoring των connections και των εντολών που εκτελούνται.

# 5. Τρόπος Λειτουργίας και Screenshots

## 5.1 Κύριο Interface της Εφαρμογής



***Screenshot 1***

Το κύριο interface της εφαρμογής (Screenshot 1) παρουσιάζει μια καθαρή και λειτουργική διεπαφή που εμφανίζει όλους τους υπολογιστές του εργαστηρίου σε μορφή λίστας. Κάθε υπολογιστής εμφανίζεται με την IP διεύθυνσή του, το hostname (όταν είναι διαθέσιμο), και έναν χρωματικό δείκτη κατάστασης. Ο πράσινος δείκτης υποδηλώνει online υπολογιστές (στο παράδειγμα ο 172.20.10.3 που τρέχει τον test server), ενώ οι κόκκινοι δείκτες υποδηλώνουν offline υπολογιστές. Η εφαρμογή εμφανίζει επιπλέον πληροφορίες όπως το λειτουργικό σύστημα για τους ενεργούς υπολογιστές και διατηρεί τα ονόματα PRPC01-PRPC27 για τους εργαστηριακούς υπολογιστές. Το "Rescan" button στο πάνω μέρος της οθόνης επιτρέπει την ανανέωση της λίστας και την επανάληψη του network scanning. Η εφαρμογή χρησιμοποιεί Material Design principles με clean typography και intuitive color coding. Η λίστα είναι scrollable και κάθε item έχει διαφορετική διαφάνεια ανάλογα με την κατάσταση, με τους offline υπολογιστές να εμφανίζονται με μειωμένη διαφάνεια για καλύτερη οπτική διαφοροποίηση.

## 1.2 Server Communication και Command Execution

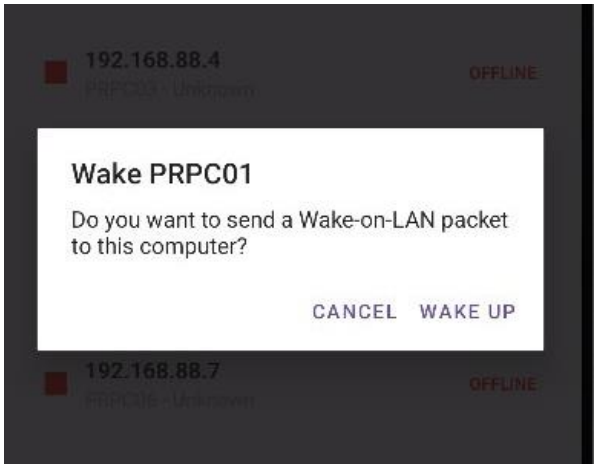
```
filip in Technology-and-Programming-of-Mobile-Devices on  main
› java Server
Server IP: 192.168.56.1
[LAPTOP-BAMKHV18] Server listening on port 41007
[LAPTOP-BAMKHV18] Received command: Echo
[LAPTOP-BAMKHV18] Received command: Echo
[LAPTOP-BAMKHV18] Received command: Restart
[LAPTOP-BAMKHV18] Received command: Shutdown
[LAPTOP-BAMKHV18] Received command: Restore
```

***Screenshot 2***

Το Screenshot 2 δείχνει την άλλη πλευρά της επικοινωνίας - τον Java server που εκτελείται σε υπολογιστή και καταγράφει τις εντολές που λαμβάνει από την Android εφαρμογή. Στο terminal βλέπουμε τη διαδικασία εκκίνησης του server όπου εμφανίζεται η IP διεύθυνση (192.168.56.1) και η επιβεβαίωση ότι ο server ακούει στην πόρτα 41007. Το hostname "LAPTOP-BAMKHV18" εμφανίζεται στα log messages που καταγράφουν κάθε εισερχόμενη εντολή. Η καταγραφή δείχνει τη σειρά εντολών που δοκιμάστηκαν: πολλαπλές "Echo" εντολές κατά τη διάρκεια του network scanning, ακολουθούμενες από "Restart", "Shutdown" και "Restore" εντολές που στάλθηκαν από την Android εφαρμογή. Κάθε εντολή καταγράφεται με timestamp και το hostname του server, παρέχοντας πλήρη ιχνηλασιμότητα των λειτουργιών. Αυτό το logging είναι

ιδιαίτερα χρήσιμο για debugging και παρακολούθηση της κίνησης στο δίκτυο του εργαστηρίου.

## 5.3 Wake-on-LAN Λειτουργικότητα




***Screenshot 3***

Το Screenshot 3 απεικονίζει το Wake-on-LAN dialog που εμφανίζεται όταν ο χρήστης πατήσει σε έναν offline υπολογιστή από τη λίστα. Το dialog εμφανίζει το μήνυμα "Wake PRPC01" με την ερώτηση "Do you want to send a Wake-on-LAN packet to this computer?" και προσφέρει δύο επιλογές: "CANCEL" για ακύρωση και "WAKE UP" για αποστολή του WOL packet. Αυτή η διεπαφή ακολουθεί τα Android design patterns με σαφή call-to-action buttons και περιγραφικό κείμενο. Όταν ο χρήστης επιλέξει "WAKE UP", η εφαρμογή στέλνει το magic packet στο MAC address του αντίστοιχου υπολογιστή που είναι προκαθορισμένο στην ComputerList κλάση. Η λειτουργία αυτή είναι ιδιαίτερα χρήσιμη στο εργαστηριακό περιβάλλον όπου οι υπολογιστές μπορεί να είναι κλειστοί για εξοικονόμηση ενέργειας αλλά χρειάζεται να ενεργοποιηθούν απομακρυσμένα. Το interface παρέχει immediate feedback στον χρήστη και διατηρεί συνέπεια με το υπόλοιπο design της εφαρμογής.

# 6. Δοκιμές και Αποτελέσματα

## 6.1 Περιβάλλον Δοκιμών

Οι δοκιμές της εφαρμογής πραγματοποιήθηκαν σε ελεγχόμενο περιβάλλον που προσομοίωνε τις συνθήκες του εργαστηρίου. Χρησιμοποιήθηκε ένας test server που εκτελείται τοπικά για την επαλήθευση της TCP επικοινωνίας και των command responses. Η εφαρμογή δοκιμάστηκε σε Android συσκευή συνδεδεμένη στο ίδιο WiFi δίκτυο με τον test server, επιβεβαιώνοντας την λειτουργία του network discovery και της command execution. Οι δοκιμές κάλυψαν όλα τα scenarios που προβλέπει η εργασία. Echo εντολών για έλεγχο κατάστασης, execution των

Restart/Shutdown/Restore εντολών με επιβεβαίωση των server responses, και δοκιμή της Wake-on-LAN λειτουργικότητας. Τα αποτελέσματα δείχνουν ότι η εφαρμογή ανταποκρίνεται σωστά σε όλες τις απαιτήσεις και διαχειρίζεται αποδοτικά τόσο τις επιτυχείς όσο και τις αποτυχημένες connections.